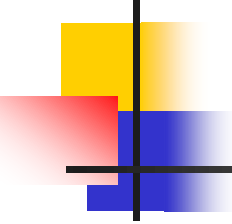


BCSE309L - Cryptography and Network Security

Module 4

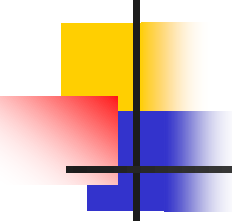
Message Digest and Hash Functions

Dr. S. Renuka Devi
Professor
SCOPE
VIT Chennai Campus



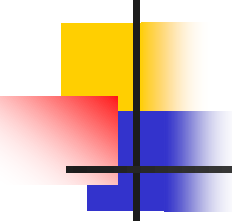
Module Outline

- Hash function - Introduction
- Requirements and security
- Secure Hash Function (SHA)
- Message Authentication Code (MAC)
- Message Digest (MD5)
- HMAC
- Birthday Attack



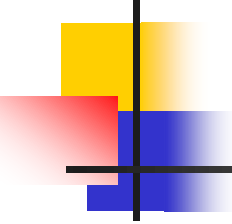
Hash functions

- Objective - Data integrity
- A **hash function** H accepts a variable-length block of data M as input and produces a fixed-size hash value $h = H(M)$
- A change to any bit or bits in M results, with high probability, in a change to the hash value



Hash function

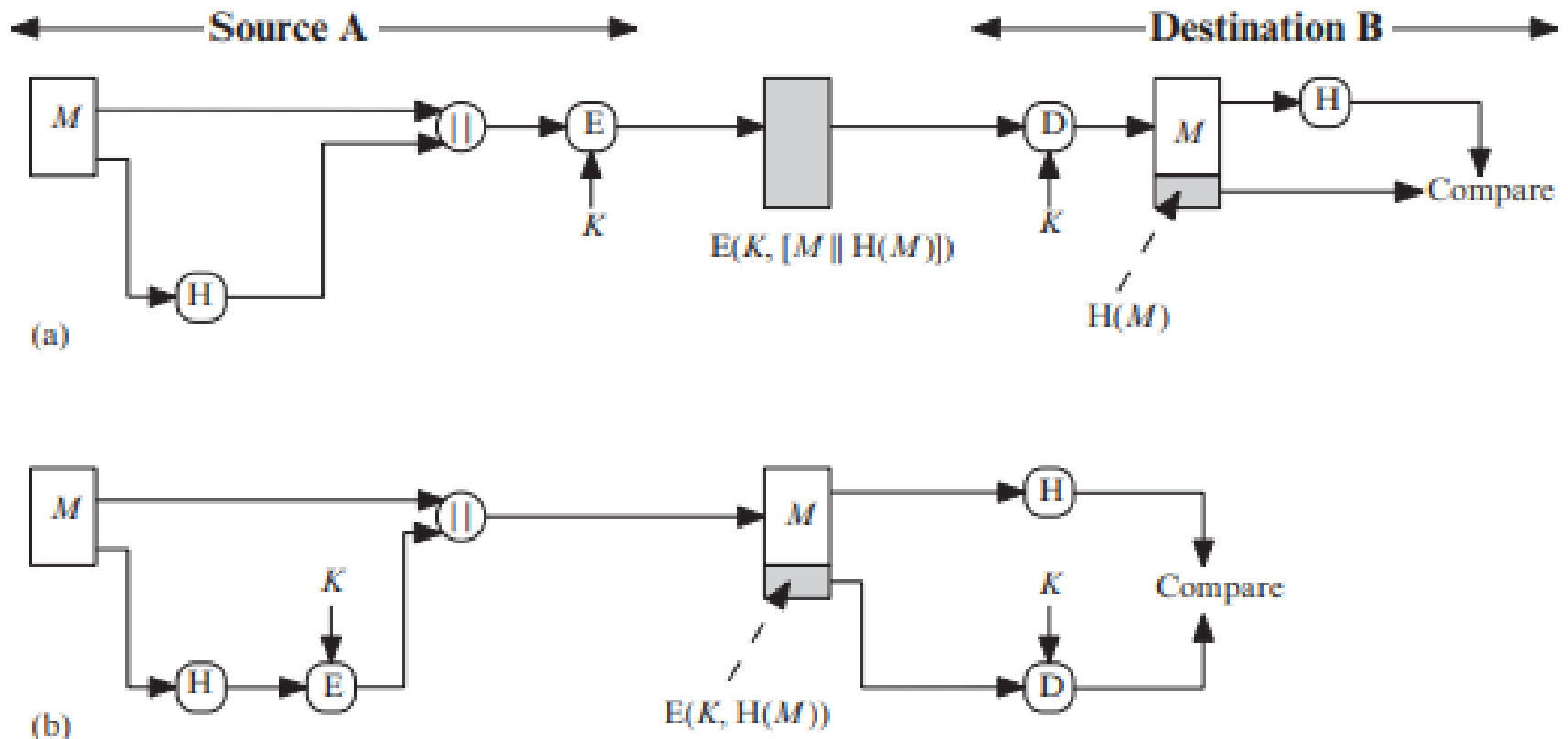
- Cryptographic hash function is an algorithm for which it is computationally infeasible to find either
 - (a) a data object that maps to a pre-specified hash result (the one-way property) or
 - (b) two data objects that map to the same hash result (collision-free property)



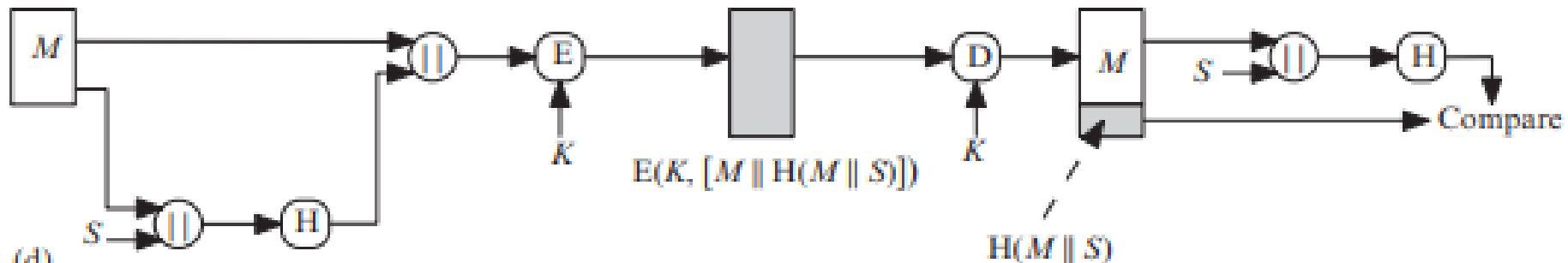
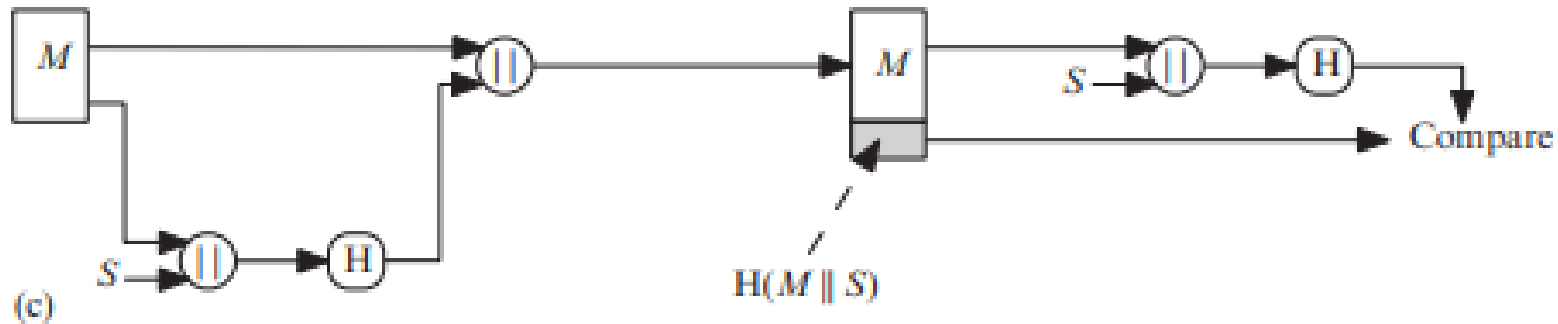
Applications of Cryptographic hash function

- Message authentication
- Digital signatures
- Other applications
 - used to create a one-way password file
 - used for intrusion detection and virus detection
 - used to construct a pseudorandom function (PRF) or a pseudorandom number generator (PRNG).

Use of a Hash Function for Message Authentication

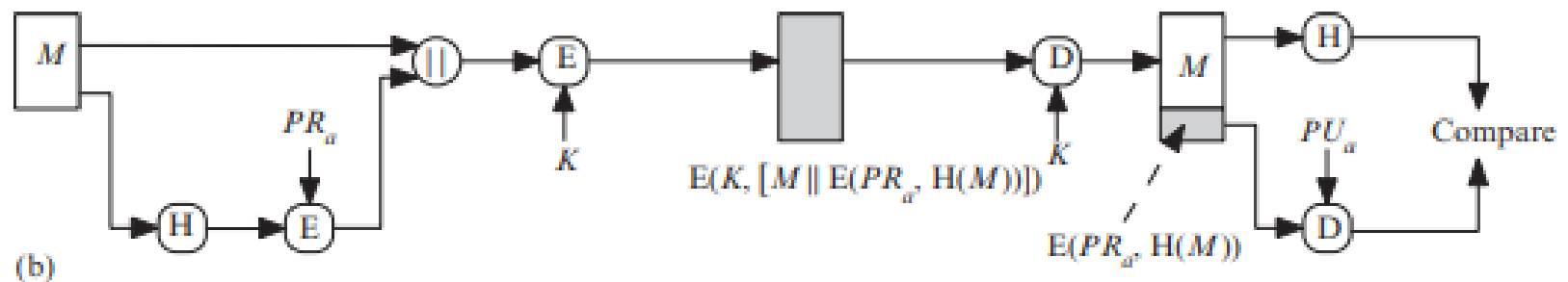
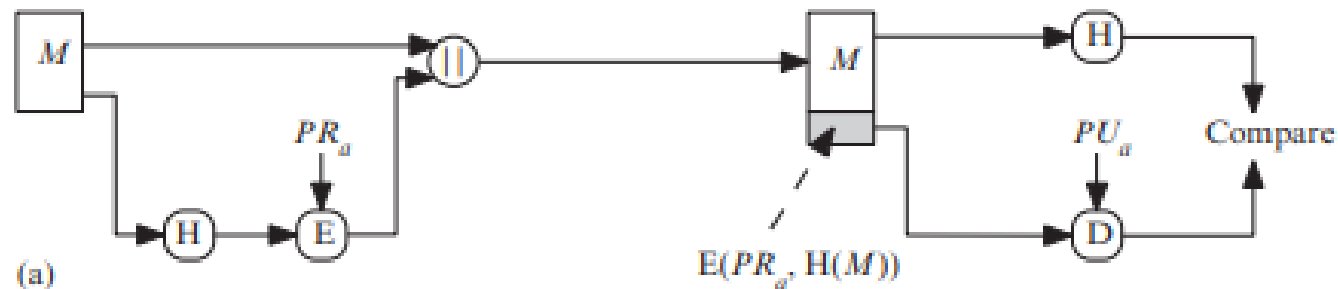


Use of a Hash Function for Message Authentication



Simplified examples of Digital Signature

← Source A → ← Destination B →



Simple Hash function

- The input is viewed as a sequence of n -bit blocks
- The input is processed one block at a time in an iterative fashion to produce an n -bit hash function
- One of the simplest hash functions is the bit-by-bit exclusive-OR (XOR) of every block. This can be expressed as

$$C_i = b_{i1} \oplus b_{i2} \oplus \cdots \oplus b_{im}$$

where

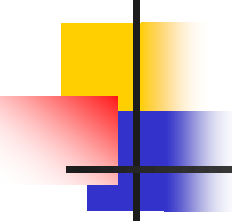
C_i = i th bit of the hash code, $1 \leq i \leq n$

m = number of n -bit blocks in the input

b_{ij} = i th bit in j th block

$\oplus$ = XOR operation

- This operation produces a simple parity bit for each bit position and is known as a **longitudinal redundancy check**.



Simple Hash function

- A simple way to improve matters is to perform a one-bit circular shift on the hash value after each block is processed.
- The procedure can be summarized as follows.
 1. Initially set the n -bit hash value to zero.
 2. Process each successive n -bit block of data as follows:
 - a. Rotate the current hash value to the left by one bit.
 - b. XOR the block into the hash value.

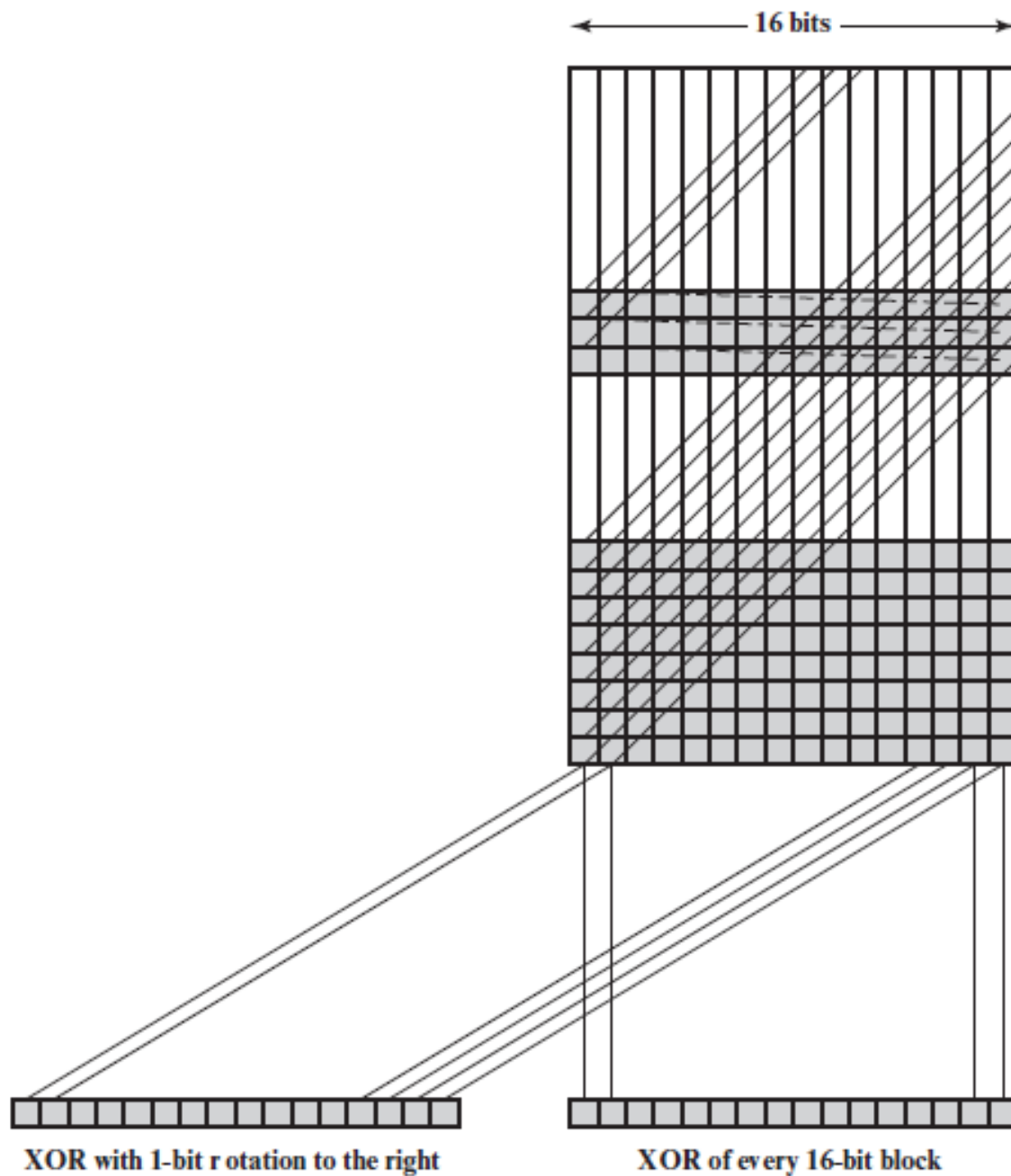
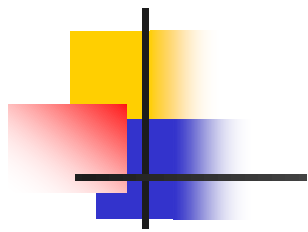
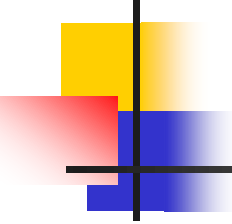


Figure 11.5 Two Simple Hash Functions

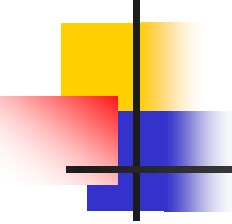


Requirements for a Cryptographic Hash Function H

- For a hash value $h = H(x)$, x is the **preimage** of h .
 - Because H is a many-to-one mapping, for any given hash value h , there will in general be multiple preimages
- A **collision** occurs if we have $x \neq y$ and $H(x) = H(y)$.

Requirements for a Cryptographic Hash Function H

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	$H(x)$ is relatively easy to compute for any given x , making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h , it is computationally infeasible to find y such that $H(y) = h$.
Second preimage resistant (weak collision resistant)	For any given block x , it is computationally infeasible to find $y \neq x$ with $H(y) = H(x)$.
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) with $x \neq y$, such that $H(x) = H(y)$.
Pseudorandomness	Output of H meets standard tests for pseudorandomness.



Hash functions based on Cipher Block Chaining

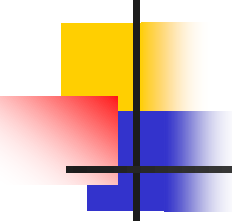
- Divide a message M into fixed-size blocks $M_1, M_2, \dots, M_N$ and use a symmetric encryption system such as DES to compute the hash code G as

$$H_0 = \text{initial value}$$

$$H_i = E(M_i, H_{i-1})$$

$$G = H_N$$

- Prone to **meet-in-the-middle-attack**
- Davies and Price [DAVI89] describe the variation:
 - $H_i = E(M_i, H_{i-1}) \oplus H_{i-1}$
- Another variation, proposed in [MEYE88], is
 - $H_i = E(H_{i-1}, M_i) \oplus M_i$



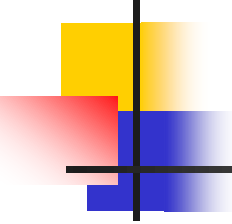
Secure Hash Algorithm (SHA)

- Most widely used hash function
- Developed by by the National Institute of Standards and Technology (NIST)
- SHA is based on the hash function MD4
- SHA-1 produces a hash value of 160 bits
- NIST produced a revised version of the standard (SHA-2), with hash value lengths of 256, 384, and 512 bits, known as SHA-256, SHA-384, and SHA-512, respectively

Table 11.3 Comparison of SHA Parameters

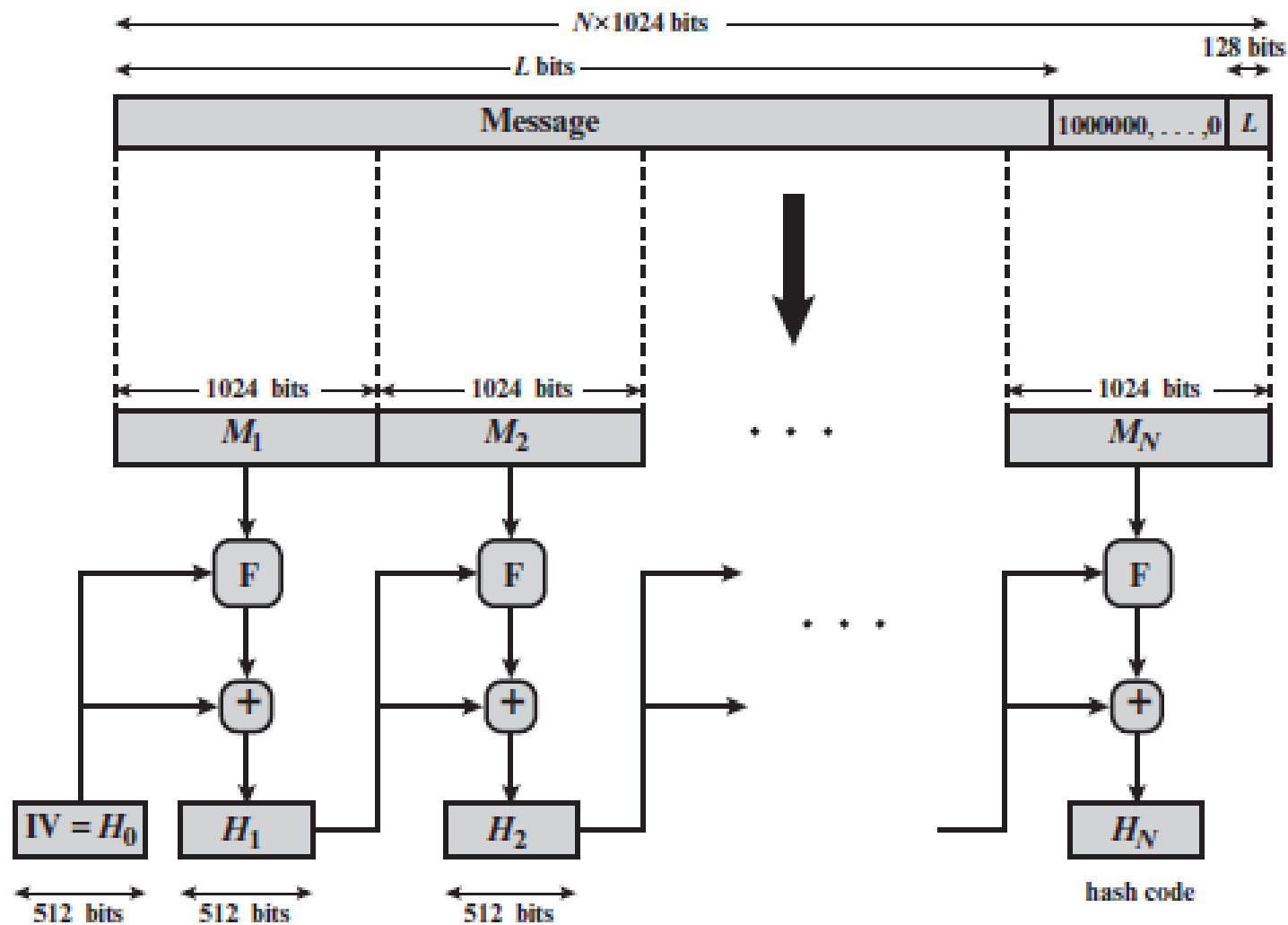
Algorithm	Message Size	Block Size	Word Size	Message Digest Size
SHA-1	$< 2^{64}$	512	32	160
SHA-224	$< 2^{64}$	512	32	224
SHA-256	$< 2^{64}$	512	32	256
SHA-384	$< 2^{128}$	1024	64	384
SHA-512	$< 2^{128}$	1024	64	512
SHA-512/224	$< 2^{128}$	1024	64	224
SHA-512/256	$< 2^{128}$	1024	64	256

Note: All sizes are measured in bits.



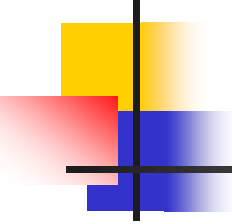
SHA-512

- The algorithm takes as input a message with a maximum length of less than 2^{128} bits and produces as output a 512-bit message digest
- The input is processed in 1024-bit blocks



$+$ = word-by-word addition mod 2^{64}

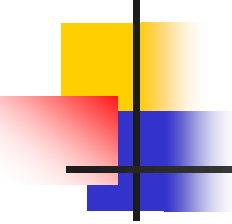
Figure 11.9 Message Digest Generation Using SHA-512



Steps involved in SHA-512

1. Append padding bits

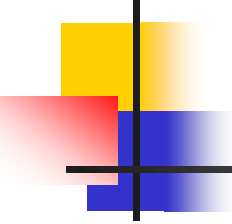
- The message is padded so that its length is congruent to 896 modulo 1024
- Padding is always added, even if the message is already of the desired length
- Thus, the number of padding bits is in the range of 1 to 1024
- The padding consists of a single 1 bit followed by the necessary number of 0 bits



Steps involved in SHA-512

2. Append length bits

- A block of 128 bits is appended to the message.
 - This block is treated as an unsigned 128-bit integer and contains the length of the original message in bits (before the padding)
-
- The outcome of the first two steps yields a message that is an integer multiple of 1024 bits in length, hence the total length of the expanded message is $N * 1024$ bits.



Steps involved in SHA-512

3. Initialize hash buffer

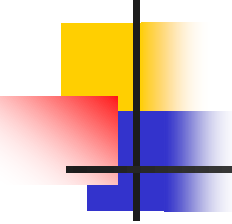
- A 512-bit buffer is used to hold intermediate and final results of the hash function
- The buffer can be represented as eight 64-bit registers (a, b, c, d, e, f, g, h)
- These registers are initialized to the following 64-bit integers (hexadecimal values):

a = 6A09E667F3BCC908 e = 510E527FADE682D1

b = BB67AE8584CAA73B f = 9B05688C2B3E6C1F

c = 3C6EF372FE94F82B g = 1F83D9ABFB41BD6B

d = A54FF53A5F1D36F1 h = 5BE0CD19137E2179



Steps involved in SHA-512

4. Process message in 1024-bit (128-byte) blocks
 - The heart of the algorithm is a module that consists of 80 rounds; this module is labeled F
 - Each round takes as input the 512-bit buffer value, abcdefgh, and updates the contents of the buffer
 - The output of the eightieth round is added to the input to the first round (H_{i-1}) to produce H_i .
 - The addition is done independently for each of the eight words in the buffer with each of the corresponding words in H_{i-1} , using addition modulo 2^{64} .

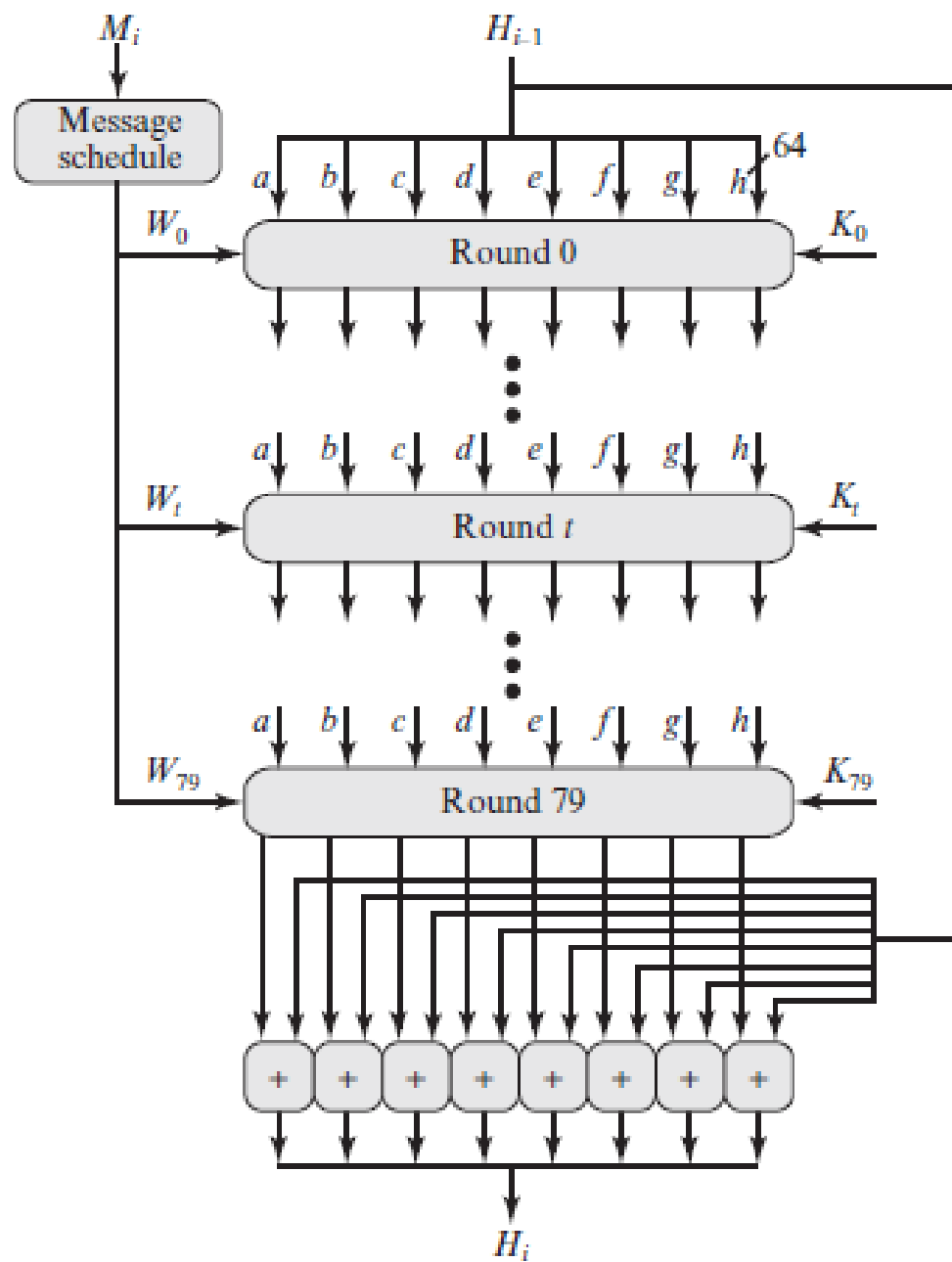
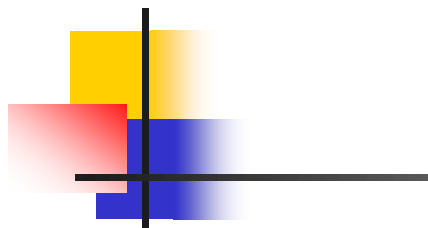
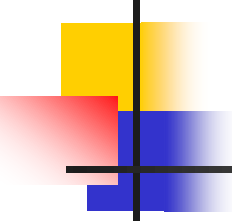


Figure 11.10 SHA-512 Processing of a Single 1024-Bit Block



Steps involved in SHA-512

5. Output

- After all N 1024-bit blocks have been processed, the output from the Nth stage is the 512-bit message digest
- The behavior of SHA-512 can be summarized as follows:

$$H_0 = IV$$

$$H_i = \text{SUM}_{64} (H_{i-1}, abcdefgh_i)$$

$$MD = H_N$$

SHA-512 Round Function

- Each round is defined by the following set of equations:

$$T_1 = h + \text{Ch}(e, f, g) + (\sum_{i=1}^{512} e) + W_t + K_t$$

$$T_2 = (\sum_{i=0}^{512} a) + \text{Maj}(a, b, c)$$

$$h = g$$

$$g = f$$

$$f = e$$

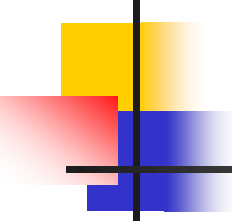
$$e = d + T_1$$

$$d = c$$

$$c = b$$

$$b = a$$

$$a = T_1 + T_2$$



SHA-512 Round Function contd...

where

t = step number; $0 \leq t \leq 79$

$\text{Ch}(e, f, g) = (e \text{ AND } f) \oplus (\text{NOT } e \text{ AND } g)$
the conditional function: If e then f else g

$\text{Maj}(a, b, c) = (a \text{ AND } b) \oplus (a \text{ AND } c) \oplus (b \text{ AND } c)$
the function is true only if the majority (two or three) of the arguments are true

$(\Sigma_0^{512} a) = \text{ROTR}^{28}(a) \oplus \text{ROTR}^{34}(a) \oplus \text{ROTR}^{39}(a)$

$(\Sigma_1^{512} e) = \text{ROTR}^{14}(e) \oplus \text{ROTR}^{18}(e) \oplus \text{ROTR}^{41}(e)$

$\text{ROTR}^n(x)$ = circular right shift (rotation) of the 64-bit argument x by n bits

SHA-512 Round Function contd...

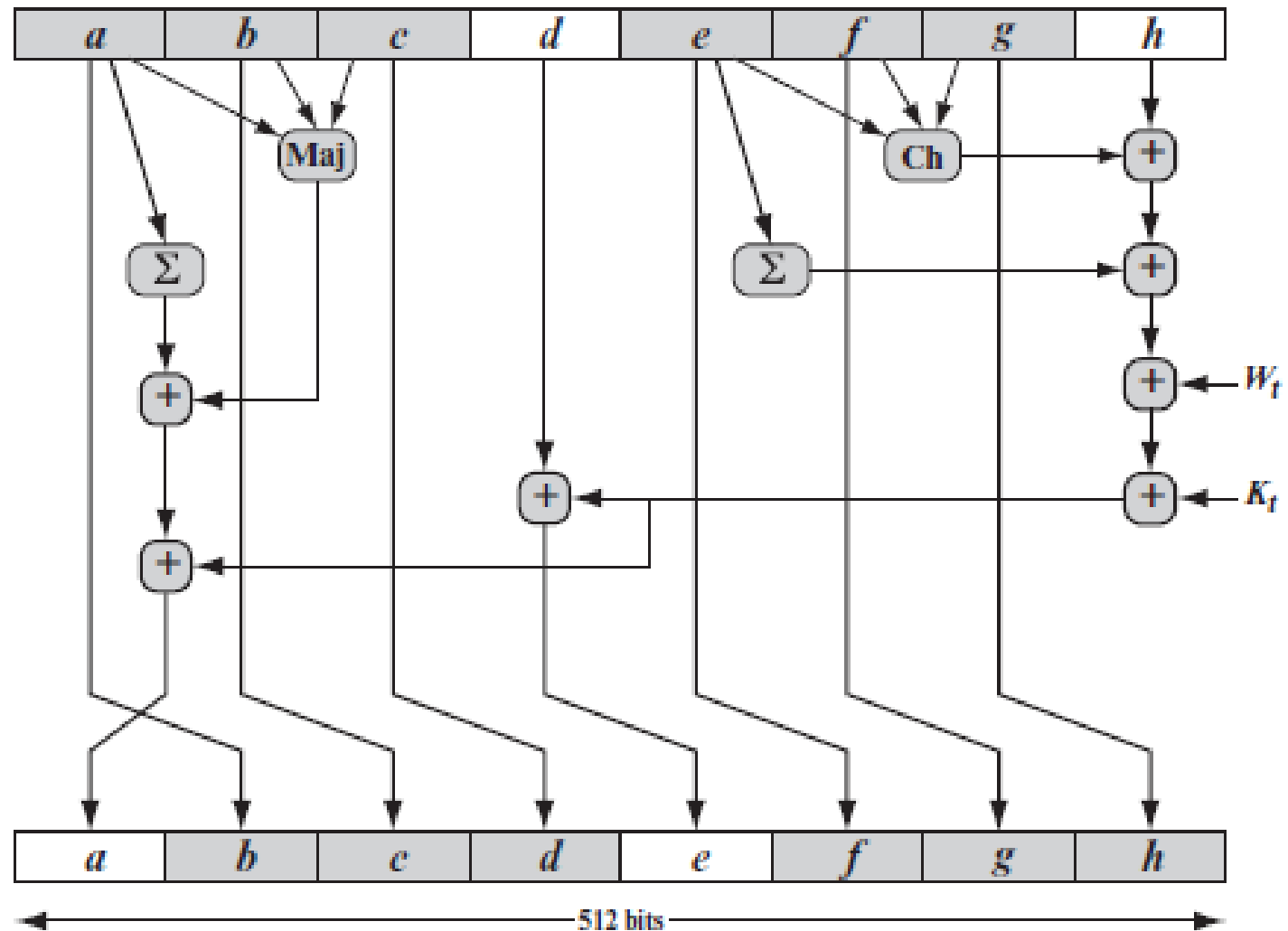
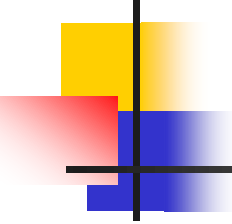


Figure 11.11 Elementary SHA-512 Operation (single round)



SHA-512 Round Function contd...

- W_t = a 64-bit word derived from the current 1024-bit input block
- K_t = a 64-bit additive constant
- $+$ = addition modulo 2^{64}

Steps to derive W_t

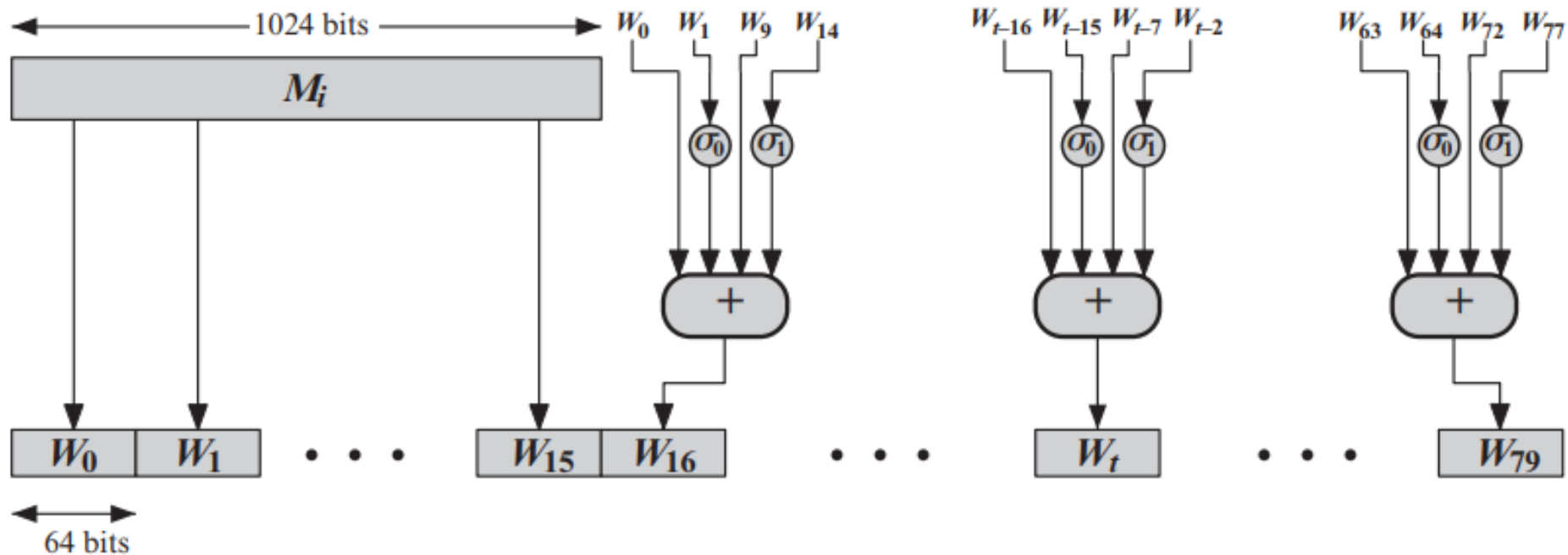
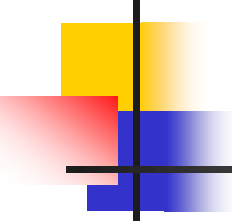


Figure 11.12 Creation of 80-word Input Sequence for SHA-512 Processing of Single Block



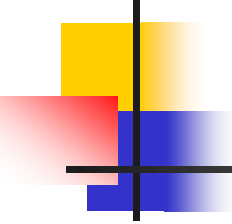
MD5 Algorithm

- used for the purpose of data verification
- produces 128bit hash value (message digest)

There are 5 prominent steps in MD5 Algorithm

1. Append padding bits

- Total number of bits must be 64 bits short from becoming the multiple of 512 (ie $448 \bmod 512$).
- In padding, we add 1 at the beginning and the rest 0's.



MD5 Algorithm

2. Append length bits

- add the length bits (64bits) to the message

3.Initialize MD buffer

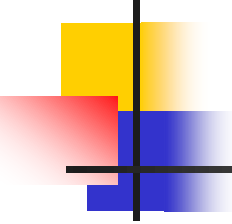
- It uses 4 buffers of 32 bits each and it is initialized as given below

A = 0, 1, 2, 3, 4, 5, 6, 7

B = 8, 9, a, b, c, d, e, f

C = f, e, d, c, b, a, 9, 8

D = 7, 6, 5, 4, 3, 2, 1, 0



MD5 Algorithm

4. Process each 512 bit block

- It consists of four rounds
- Each round is composed of 16 similar operations based on a non-linear function F , modular addition, and left rotation.
- There are four possible functions (F); a different one is used in each round:

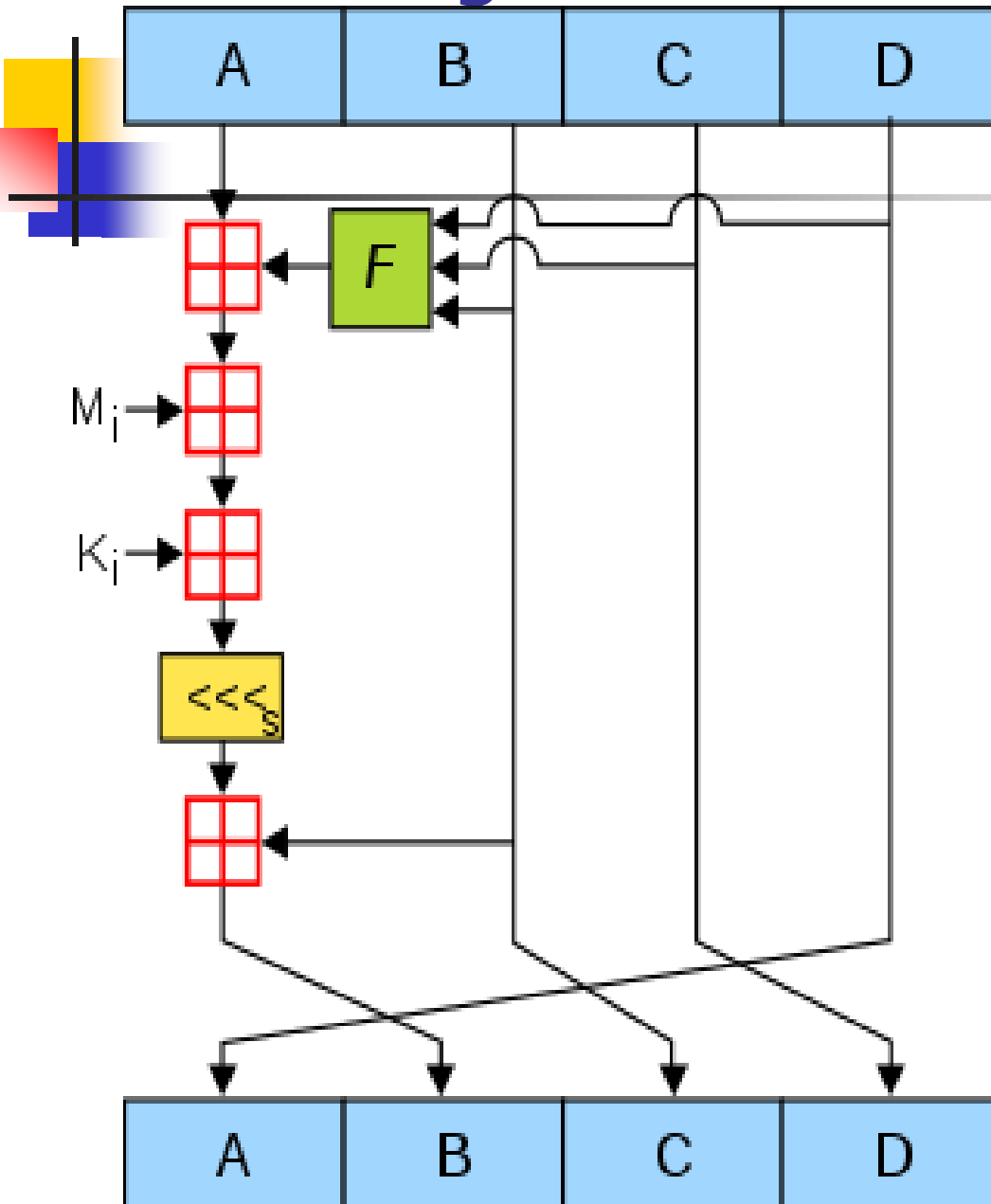
$$F(B, C, D) = (B \wedge C) \vee (\neg B \wedge D)$$

$$G(B, C, D) = (B \wedge D) \vee (C \wedge \neg D)$$

$$H(B, C, D) = B \oplus C \oplus D$$

$$I(B, C, D) = C \oplus (B \vee \neg D)$$

MD5 Algorithm



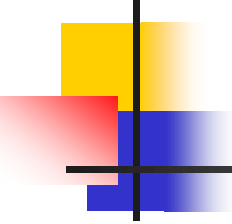
F is a nonlinear function;
one function is used in
each round (G, H, I)

M_i denotes a 32-bit block
of the message input

K_i denotes a 32-bit
constant, different for
each operation

$\lll_s$ denotes a left bit
rotation by s places;
 s varies for each
operation

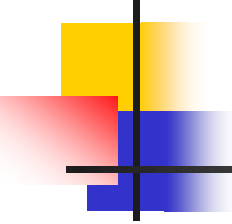
 denotes addition
modulo 2^{32} .



MD5 Algorithm

5. Output Message Digest

- The output of the last 512 bit block is ultimately our message digest.



Message Authentication Code (MAC)

- It involves the use of a secret key to generate a small fixed-size block of data, known as a **cryptographic checksum** or **MAC**
- This technique assumes that two communicating parties, say A and B, share a common secret key.
- When A has a message to send to B, it calculates the MAC as a function of the message and the key:

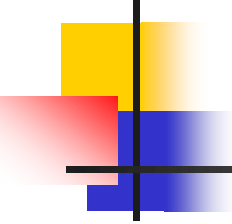
$$\text{MAC} = C(K, M)$$

Where M - input message

C - MAC function

K - shared secret key

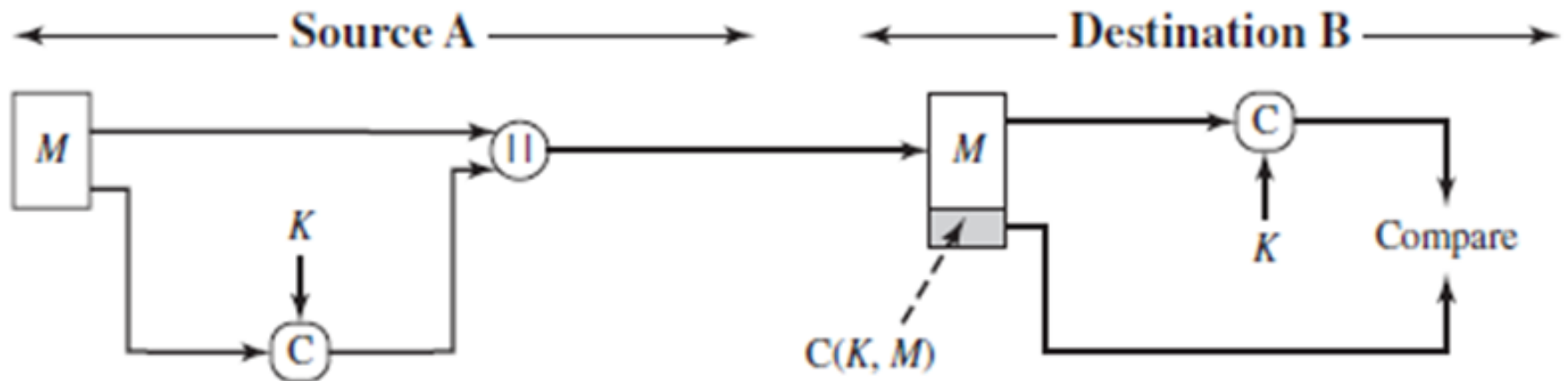
MAC - message authentication code



MAC

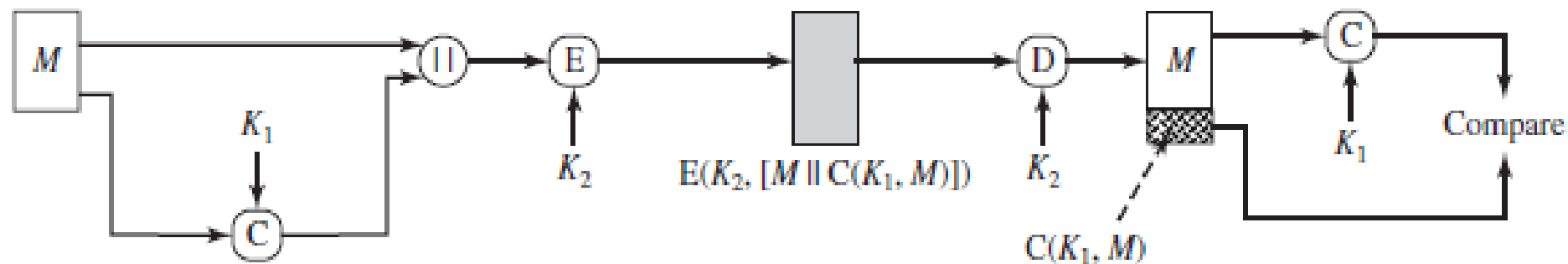
- The message plus MAC are transmitted to the intended recipient
- The recipient performs the same calculation on the received message, using the same secret key, to generate a new MAC
- If the received MAC matches the calculated MAC, then
 - The receiver is assured that the message has not been altered
 - The receiver is assured that the message is from the alleged sender

Basic Uses of MAC

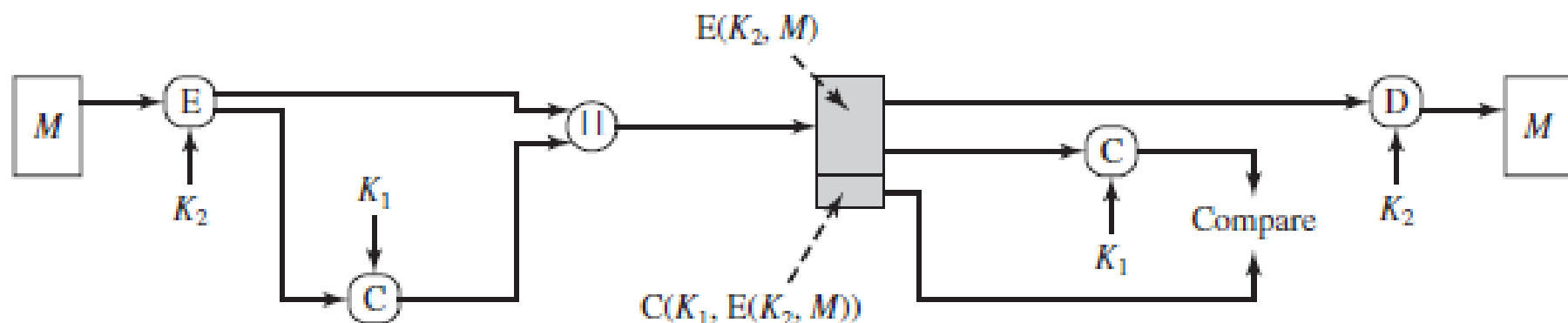


(a) Message authentication

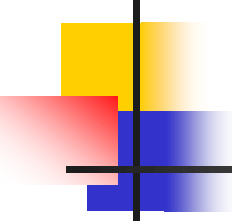
Basic Uses of MAC



(b) Message authentication and confidentiality; authentication tied to plaintext

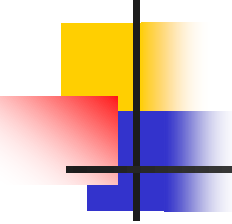


(c) Message authentication and confidentiality; authentication tied to ciphertext



Requirements of MAC

1. If an opponent observes M and $\text{MAC}(K, M)$, it should be computationally infeasible for the opponent to construct a message such that $\text{MAC}(K, M') = \text{MAC}(K, M)$
2. $\text{MAC}(K, M)$ should be uniformly distributed in the sense that for randomly chosen messages, M and M' , the probability that $\text{MAC}(K, M') = \text{MAC}(K, M)$ is 2^{-n} , where n is the number of bits in the tag.
3. Let M' be equal to some known transformation on M . That is, $M' = f(M)$. In that case,
 $\Pr[\text{MAC}(K, M') = \text{MAC}(K, M)] = 2^{-n}$

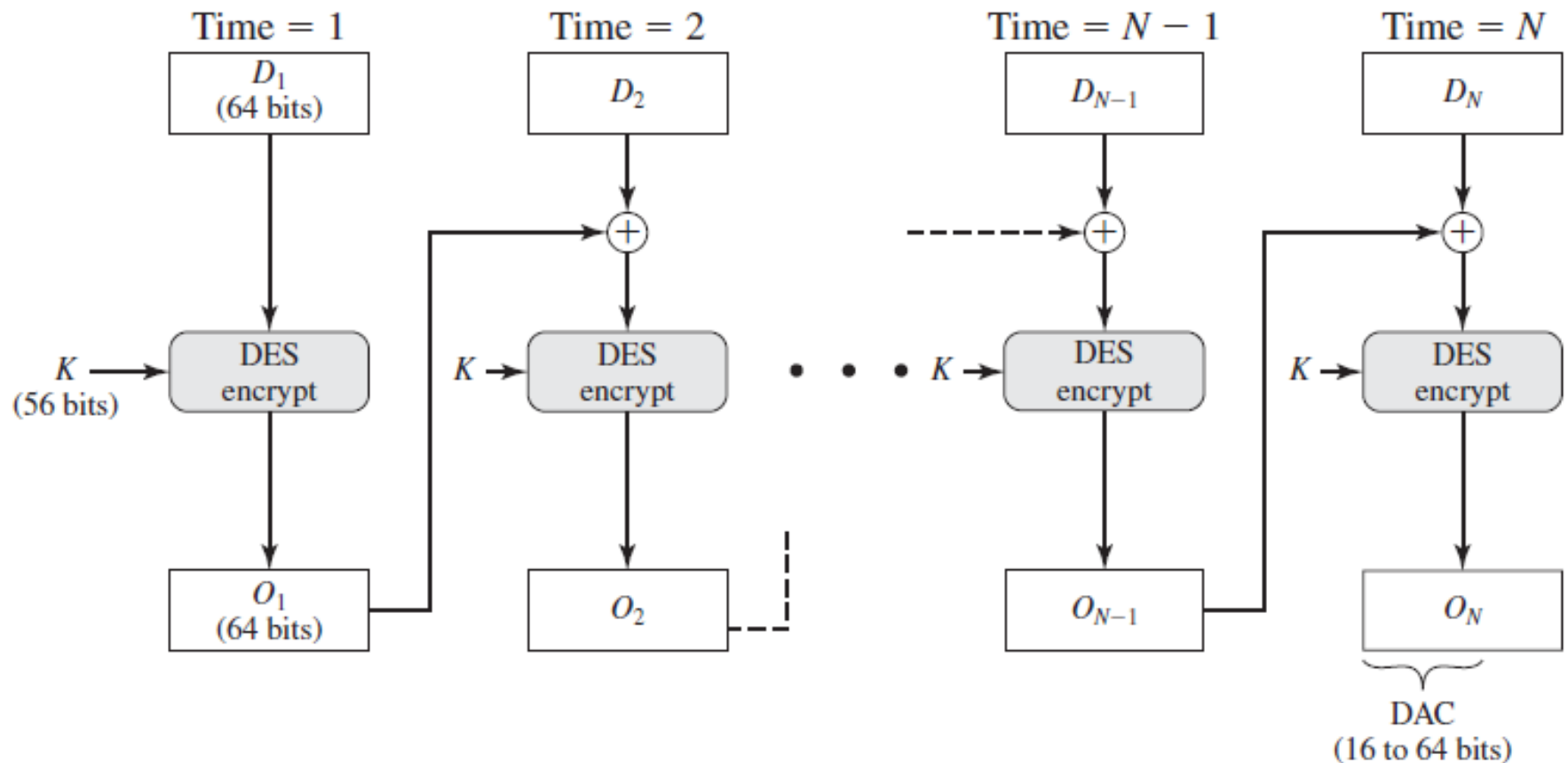


Security of MAC

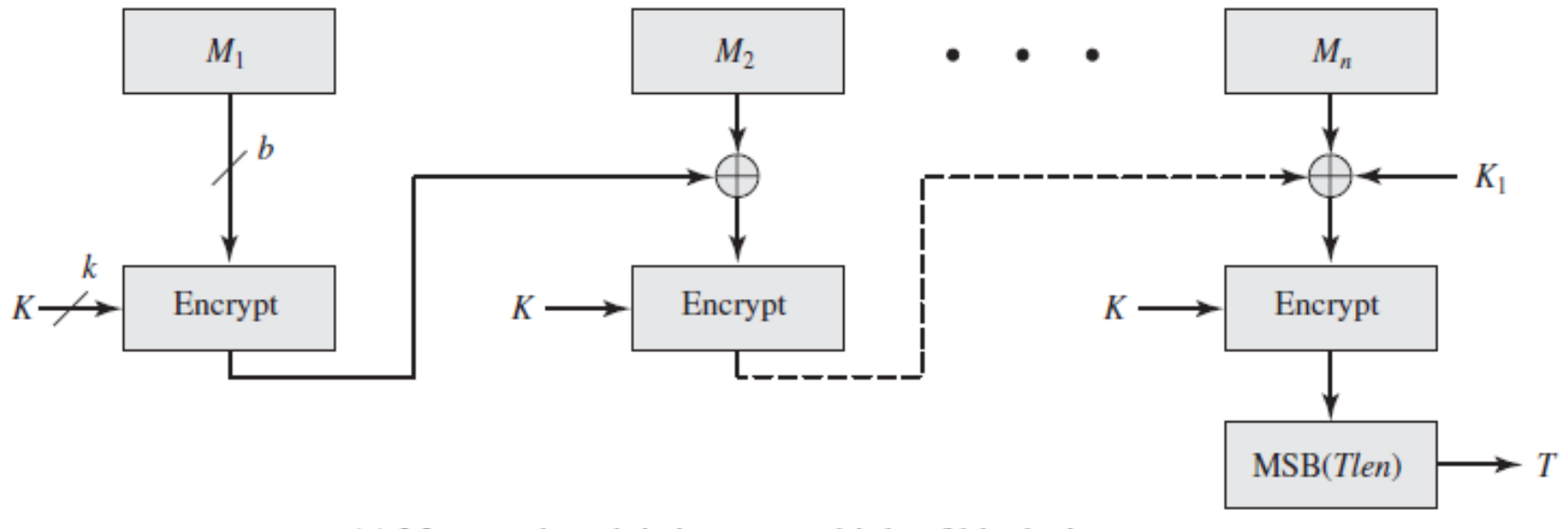
The desired security property of a MAC algorithm is expressed as follows.

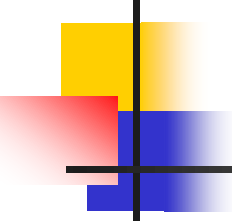
- Computation resistance: Given one or more text-MAC pairs $[x_i, \text{MAC}(K, x_i)]$, it is computationally infeasible to compute any text-MAC pair $[x, \text{MAC}(K, x)]$, for any new input $x \neq x_i$.

MAC based on block Ciphers: Data Authentication Algorithm (DAA)



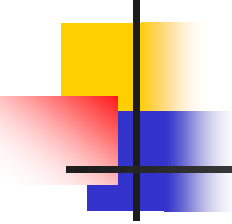
MACS BASED ON BLOCK CIPHERS: Cipher based Message Authentication Code(CMAC)





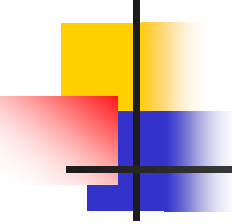
MACs based on Hash Function: HMAC

- The motivations behind developing a MAC derived from a Cryptographic hash function are
 1. Cryptographic hash functions such as MD5 and SHA generally execute faster in software than symmetric block ciphers such as DES.
 2. Library code for cryptographic hash functions is widely available.



HMAC Algorithm

- H = embedded hash function (e.g., MD5, SHA-1, RIPEMD-160)
- IV = initial value input to hash function
- M = message input to HMAC (including the padding specified in the embedded hash function)
- Y_i = i th block of M , $0 \dots i \dots (L - 1)$
- L = number of blocks in M



HMAC Algorithm

- b = number of bits in a block
- n = length of hash code produced by embedded hash function
- K = secret key; recommended length is $\geq n$; if key length is greater than b , the key is input to the hash function to produce an n -bit key
- K^+ = K padded with zeros on the left so that the result is b bits in length

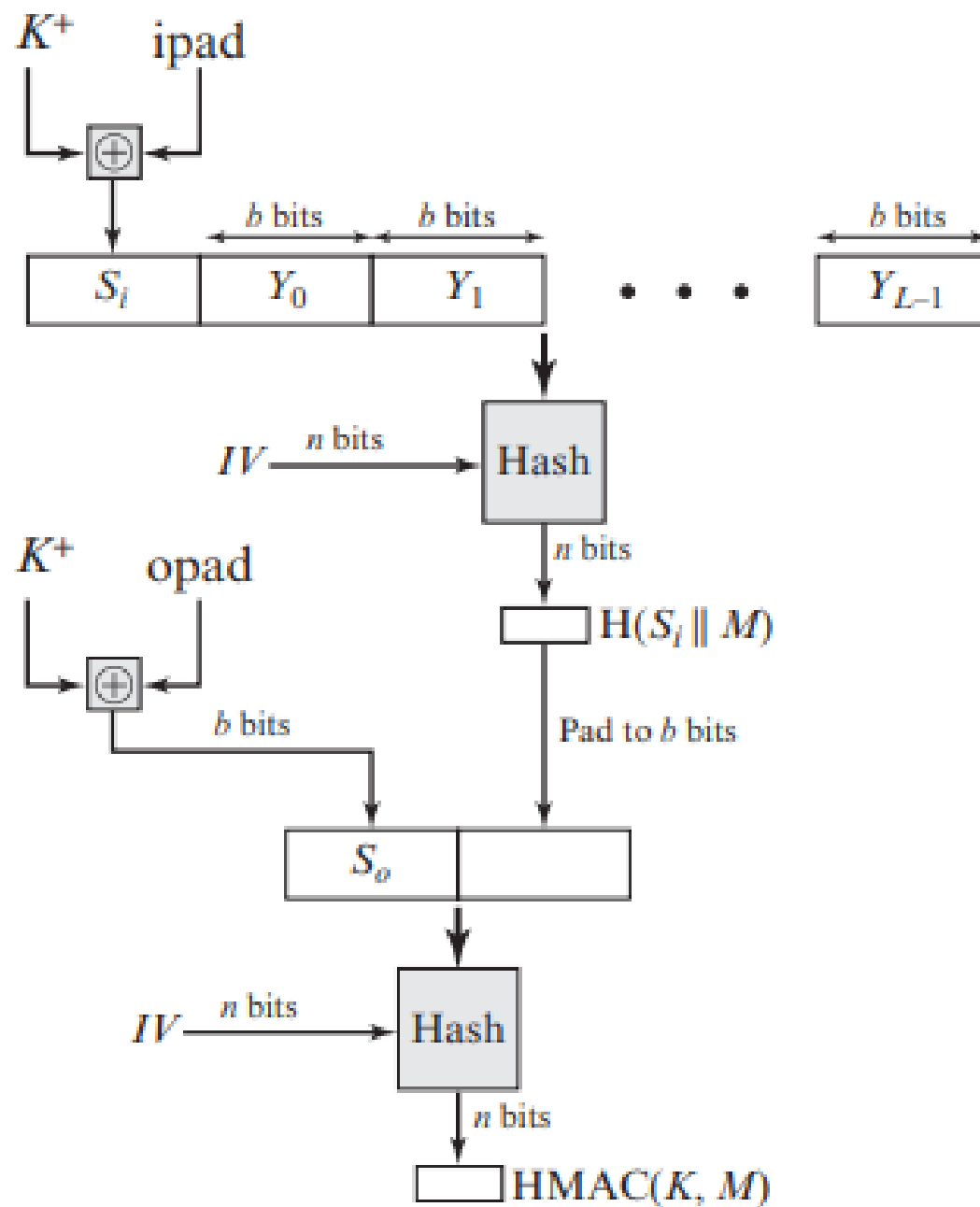
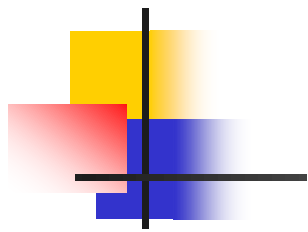
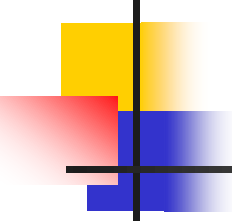


Figure 12.5 HMAC Structure

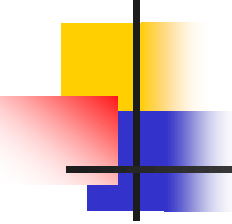


HMAC Algorithm

- ipad = 00110110 (36 in hexadecimal) repeated $b/8$ times
- opad = 01011100 (5C in hexadecimal) repeated $b/8$ times

Then HMAC can be expressed as

$$\text{HMAC}(K, M) = H[(K^+ \oplus \text{opad}) \parallel H[(K^+ \oplus \text{ipad}) \parallel M]]$$



HMAC Algorithm

1. Append zeros to the left end of K to create a b -bit string K^+ (e.g., if K is of length 160 bits and $b = 512$, then K will be appended with 352 bits of 0).
2. XOR (bitwise exclusive-OR) K^+ with ipad to produce the b -bit block S_i .
3. Append M to S_i .
4. Apply H to the stream generated in step 3.
5. XOR K^+ with opad to produce the b -bit block S_0 .
6. Append the hash result from step 4 to S_0 .
7. Apply H to the stream generated in step 6 and output the result

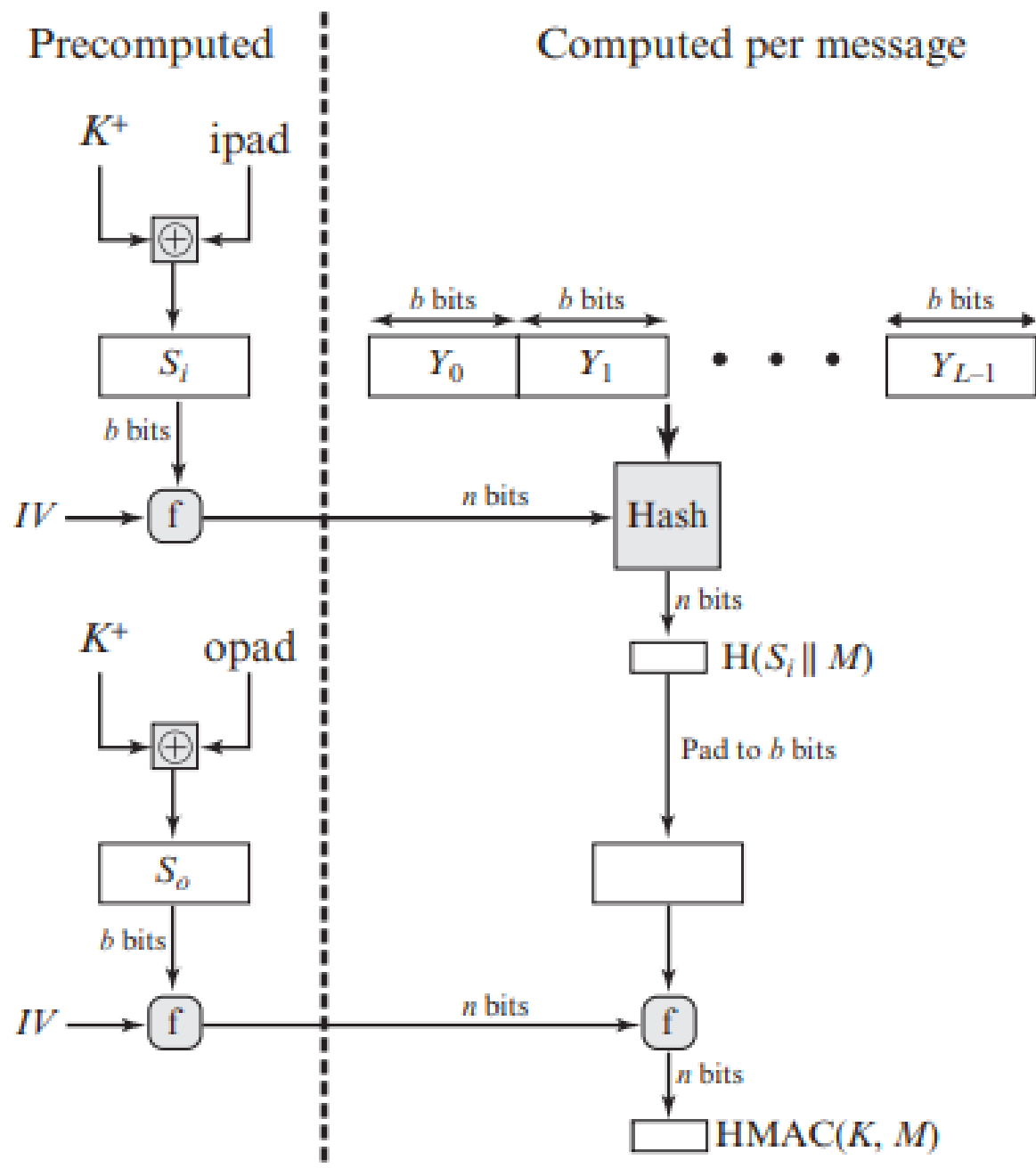
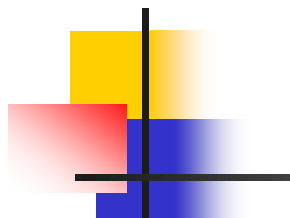
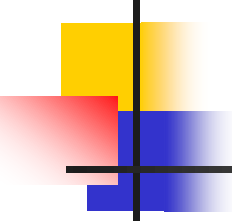


Figure 12.6 Efficient Implementation of HMAC

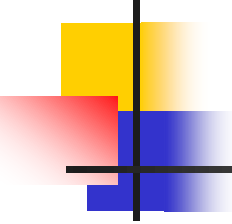


Birthday Attack

- It belongs to a class of brute force attacks
- It exploits the mathematics behind the birthday problem in probability theory
- The success of this attack largely depends upon the higher likelihood of collisions found between random attack attempts and a fixed degree of permutations
- With a birthday attack, it is possible to find a collision of a hash function with 50% chance in $2^{n/2}$

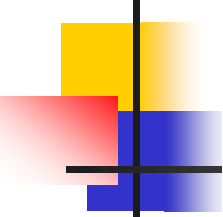
Birthday Attack

1. The source, A, is prepared to sign a legitimate message x by appending the appropriate m -bit hash code and encrypting that hash code with A's private key
2. The opponent generates $2^{m/2}$ variations x' of x , all of which convey essentially the same meaning, and stores the messages and their hash values
3. The opponent prepares a fraudulent message y for which A's signature is desired
4. The opponent generates minor variations y' of y , all of which convey essentially the same meaning. For each y' , the opponent computes $H(y')$, checks for matches with any of the $H(x')$ values, and continues until a match is found.



Birthday Attack

5. The opponent offers the valid variation to A for signature.
 6. This signature can then be attached to the fraudulent variation for transmission to the intended recipient. Because the two variations have the same hash code, they will produce the same signature; the opponent is assured of success even though the encryption key is not known
- Thus, if a 64-bit hash code is used, the level of effort required is only on the order of 2^{32} .



{ This letter is } to introduce { you to } { Mr. } Alfred { P. }
 { I am writing } { to you } { -- }

Barton, the { new } { chief } jewellery buyer for { our }
 { newly appointed } { senior } { the }

Northern { European } { area } . He { will take } over { the }
 { Europe } { division } { has taken } { -- }

responsibility for { all } our interests in { watches and jewellery }
 { the whole of } { jewellery and watches }

in the { area } . Please { afford } him { every } help he { may need }
 { region } { give } { all the } { needs }

to { seek out } the most { modern } lines for the { top } end of the
 { find } { up to date } { high }

market. He is { empowered } to receive on our behalf { samples } of the
 { authorized } { specimens }

{ latest } { watch and jewellery } products, { up } to a { limit }
 { newest } { jewellery and watch } { subject } { maximum }

of ten thousand dollars. He will { carry } a signed copy of this { letter }
 { hold } { document }

as proof of identity. An order with his signature, which is { appended }
 { attached }

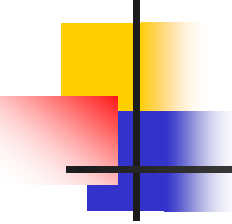
{ authorizes } you to charge the cost to this company at the { above }
 { allows } { head office }

address. We { fully } expect that our { level } of orders will increase in
 { -- } { volume }

the { following } year and { trust } that the new appointment will { be }
 { next } { hope } { prove }

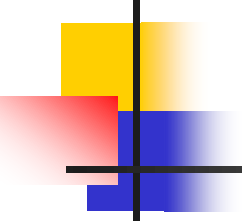
{ advantageous } to both our companies.
 { an advantage }

Figure 11.6 A Letter in 2³⁷ Variation [DAVI89]



References

- Cryptography and Network Security Principles And Practice, William Stallings, 5e , Pearson Education, 2011



Thank you
